

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Department of Computer Science - Software Engineering

ADVANCED WEB DEVELOPMENT

Names: American Latin Class Team

NRC: 30716

Date: May 14, 2026

American Latin Class - Test Cases Report

Product Backlog Decomposition into Features by Module

Introduction

This document serves as the Quality Assurance (QA) test cases report for the American Latin Class web platform. The tests verify the implementation of the user stories, functional requirements, and features defined in the project backlog.

The system includes a public website, enrollment flow, role-based portal, student management, attendance control, teacher planning, branch finance reports, professional B2 dancer events, and audit/security behavior.

Test Execution Summary

Feature ID	Feature	Total Test Cases	Functional	Pending / Review
FT-01	Public Landing and Branch Information	6	6	0
FT-02	Public Enrollment Request	6	6	0
FT-03	Authentication and Role-Based Access	6	6	0
FT-04	Student Management, Levels, and Scholarships	6	6	0
FT-05	Teacher Monthly Class Planning	6	6	0
FT-06	Attendance Management and Kiosk Check-In	6	6	0
FT-07	Branch Finance and Matrix Share	6	6	0
FT-08	Professional B2 Events and Dancer Settlements	6	6	0
FT-09	Audit, Security, Deployment, and API Health	6	6	0
Total		**54**	**54**	**0**

Feature FT-01: Public Landing and Branch Information

As a visitor, I want to view academy information and available branches without logging in.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-01	Open public home page	1. Open index.html or the Netlify home URL.	The American Latin Class landing page loads successfully.	Functional
CP-ALC-02	View academy information	1. Open the home page. 2. Review sections about programs and academy purpose.	Public academy information is visible without authentication.	Functional
CP-ALC-03	Load branch list	1. Open a page that uses branch data. 2. Allow the frontend to call /api/branches.	Branch options are loaded from the backend or fallback options appear if the API is unavailable.	Functional
CP-ALC-04	Public navigation	1. Use navigation links from the public site.	The user can reach enrollment, attendance kiosk, and login pages.	Functional
CP-ALC-05	Backend branch endpoint	1. Send GET /api/branches.	The API returns a JSON response with available branches.	Functional
CP-ALC-06	Mobile responsive view	1. Open the home page on a small viewport.	Content remains readable and usable on mobile screens.	Functional

Feature FT-02: Public Enrollment Request

As a visitor, I want to submit an enrollment request so the academy can review my information.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-07	Valid enrollment submission	1. Open enrollment.html. 2. Complete all required fields. 3. Submit the form.	The backend creates a pending student enrollment request.	Functional
CP-ALC-08	Empty enrollment form	1. Open the enrollment form. 2. Submit without required data.	Required field validation blocks the request or returns validation errors.	Functional
CP-ALC-09	Invalid email	1. Enter an invalid email value. 2. Submit.	The API returns an email validation error.	Functional
CP-ALC-10	Invalid national ID length	1. Enter a national ID shorter than 6 digits or longer than 20 digits. 2. Submit.	The API rejects the national ID length.	Functional
CP-ALC-11	Duplicate enrollment data	1. Submit an enrollment using an existing national ID, email, or phone.	The API rejects the duplicate request with a clear message.	Functional
CP-ALC-12	Invalid branch ID	1. Send an enrollment request with a branch ID that does not exist.	The API returns 422 and does not create the student.	Functional

Feature FT-03: Authentication and Role-Based Access

As an internal user, I want to log in securely and access only the modules allowed for my role.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-13	Valid login	1. Open login.html. 2. Enter valid credentials. 3. Submit.	The backend returns a signed token and user information.	Functional
CP-ALC-14	Invalid login	1. Enter a valid email with an incorrect password. 2. Submit.	The API returns 401 with an invalid credentials message.	Functional
CP-ALC-15	Empty login form	1. Submit login without email or password.	The API returns validation error 422.	Functional
CP-ALC-16	Protected route without token	1. Send GET /api/students without Authorization.	The API returns 401 Authentication required.	Functional
CP-ALC-17	Forbidden role access	1. Log in as a student. 2. Try to access director-only endpoints.	The API returns 403 for unauthorized role access.	Functional
CP-ALC-18	Session cleared on unauthorized response	1. Use an invalid or expired token from the frontend. 2. Trigger a protected request.	The frontend clears the session and forces the user to log in again.	Functional

Feature FT-04: Student Management, Levels, and Scholarships

As a director, I want to review students by branch, level, and scholarship status.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-19	Director student list	1. Log in as director. 2. Open the students module.	The dashboard displays student records.	Functional
CP-ALC-20	Student branch information	1. Review the students table.	Each student displays branch information.	Functional
CP-ALC-21	B1/B2 level display	1. Review the students table.	Student level appears as B1 or B2.	Functional
CP-ALC-22	Scholarship percentage display	1. Review scholarship data in the students table.	Scholarship is shown as 0%, 50%, 75%, or 100%.	Functional
CP-ALC-23	Branch-scoped director access	1. Log in as a branch director. 2. Request another branch.	The API prevents access to branches outside the user's scope.	Functional
CP-ALC-24	Parent-specific payment reminders	1. Log in as a parent account. 2. Review payment reminders.	Parent payment reminder module should be available.	Functional

Feature FT-05: Teacher Monthly Class Planning

As a teacher, I want to submit monthly class plans for my branch and level.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-25	Submit valid class plan	1. Log in as teacher. 2. Complete teacher, month, level, objective, and activities. 3. Submit.	The API creates a class plan with status submitted.	Functional
CP-ALC-26	Missing class plan fields	1. Submit a class plan with empty required fields.	The API returns validation errors.	Functional
CP-ALC-27	Teacher own branch write	1. Log in as teacher. 2. Submit a plan for the teacher branch.	The plan is accepted.	Functional
CP-ALC-28	Teacher wrong branch write	1. Log in as teacher. 2. Submit a plan for another branch.	The API returns 403.	Functional
CP-ALC-29	Director creates branch plan	1. Log in as matrix director. 2. Submit a plan for any branch.	The plan is accepted for the selected branch.	Functional
CP-ALC-30	Strict month format validation	1. Submit a class plan with invalid month format.	The API should reject invalid month formats.	Functional

Feature FT-06: Attendance Management and Kiosk Check-In

As a teacher or student, I want attendance to be registered and reviewed correctly.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-31	Manual student attendance	1. Log in as teacher or director. 2. Submit student attendance.	The API creates an attendance record with source manual.	Functional
CP-ALC-32	Manual teacher attendance	1. Submit attendance with person type teacher.	The API accepts teacher attendance records.	Functional
CP-ALC-33	Invalid attendance status	1. Submit status outside present, absent, late, or excused.	The API returns a validation error.	Functional
CP-ALC-34	Kiosk valid check-in	1. Open attendance-kiosk.html. 2. Enter an active student's national ID.	The API creates a kiosk attendance record and returns an evidence code.	Functional
CP-ALC-35	Kiosk duplicate same day	1. Register the same student twice on the same day.	The second request returns the existing attendance record instead of creating a duplicate.	Functional
CP-ALC-36	Student monthly attendance view	1. Log in as student. 2. Open monthly attendance module.	Attendance records and summary counters are displayed.	Functional

Feature FT-07: Branch Finance and Matrix Share

As a director, I want to register branch income and expenses and calculate the matrix share.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-37	Create valid finance report	1. Log in as director. 2. Enter branch, income, expenses, and matrix share percent. 3. Submit.	The API creates a finance report.	Functional
CP-ALC-38	Matrix share calculation	1. Submit income and matrix share percent.	matrix_share_amount is calculated from income and percentage.	Functional
CP-ALC-39	Net result calculation	1. Submit income, expenses, and matrix share percent.	net_result equals income minus expenses minus matrix share.	Functional
CP-ALC-40	Negative finance values	1. Submit negative income, expenses, or matrix share percent.	The API rejects negative values.	Functional
CP-ALC-41	Matrix share over 100 percent	1. Submit matrix share percent greater than 100.	The API returns a validation error.	Functional
CP-ALC-42	Finance charts for branch comparison	1. Open dashboard finance charts.	Visual comparison charts should be displayed.	Functional

Feature FT-08: Professional B2 Events and Dancer Settlements

As a director, I want to register professional events and calculate dancer payments.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-43	Create professional event	1. Log in as director. 2. Submit client, event type, date, amount, and status.	The API creates the professional event.	Functional
CP-ALC-44	Reject negative event amount	1. Submit a professional event with negative total amount.	The API returns a validation error.	Functional
CP-ALC-45	Assign B2 dancer	1. Create an event. 2. Assign an active B2 student.	The API creates a dancer assignment.	Functional
CP-ALC-46	Reject non-B2 dancer	1. Attempt to assign a B1 student.	The API rejects the assignment.	Functional
CP-ALC-47	Dancer settlement calculation	1. Request /api/dancer-settlements/{studentid}.	The API returns gross amount, deductions, net amount, and assignments.	Functional
CP-ALC-48	Event commission formula validation	1. Apply a configurable commission rule per event.	The system should support owner-approved commission formulas.	Functional

Feature FT-09: Audit, Security, Deployment, and API Health

As an administrator or reviewer, I want protected operations to be auditable and the deployed system to be verifiable.

ID	Test Case	Steps	Expected Result	Status
CP-ALC-49	API health check	1. Send GET /api/health.	The API returns status ok and database connected when configured.	Functional
CP-ALC-50	Railway backend deployment	1. Open the Railway backend URL. 2. Check /api/health.	The deployed backend responds successfully.	Functional
CP-ALC-51	Netlify frontend deployment	1. Open the Netlify URL.	The deployed frontend loads successfully.	Functional
CP-ALC-52	Protected write audit log	1. Submit class plan, attendance, finance, or event data through a protected route.	The system records an audit entry when the audit table is available.	Functional
CP-ALC-53	Sensitive writes through backend	1. Review frontend requests.	The browser calls the PHP API and does not write sensitive records directly to Supabase.	Functional
CP-ALC-54	Secure production error message	1. Trigger a database connection failure.	Public API responses use a generic error message instead of exposing raw credentials or SQL details.	Functional

Conclusion

The project satisfies the requirement of having at least nine features. This report documents nine feature modules and fifty-four test cases. All documented cases are currently functional after the final verification pass.

The active architecture follows MVC through 06Code/Model, 06Code/View, and 06Code/Controller, with object-oriented services and validators supporting clean separation of responsibilities.